

-
- 使用前注意事项
 - 一、FlinkKafkaProducer的构造函数使用
 - 二、有关kafka参数和RetryableProducerCallback参数规范
- 数据完整性测试
 - 一、前期数据准备
 - 二、启动Job 开始模拟Flink正常运行的任务
 - 三、关闭broker2
 - 四、屏蔽坏partition后稳定状态
 - 五、恢复broker2的kafka进程

使用前注意事项

一、FlinkKafkaProducer的构造函数使用

目前FlinkKafkaProducer的构造函数主要提供了两种Schema的模式，主要分为**KafkaSerializationSchema**和**KafkaSerializationSchemaWrapper**这两种模式。具体反映到代码中，针对这两种模式有着不同的处理方式。目前自定义的FlinkKafkaProducer同时支持这两种。

主要构造函数如下。

```
// 目前项目主要使用的模式，精简了一些参数的设置，数据对象为KafkaSerializationSchema
public static FlinkKafkaProducer<byte[]> createKafkaProducer(Properties kafkaProps, String outputTopic, FlinkKafkaProducer<byte[]> sink = new FlinkKafkaProducer<>(outputTopic,
    (KafkaSerializationSchema<byte[]>) (element, timestamp) -> new ProducerRecord<>(outputTopic, null /* robin */, timestamp, kafkaProps, semantic);
    return sink;
}

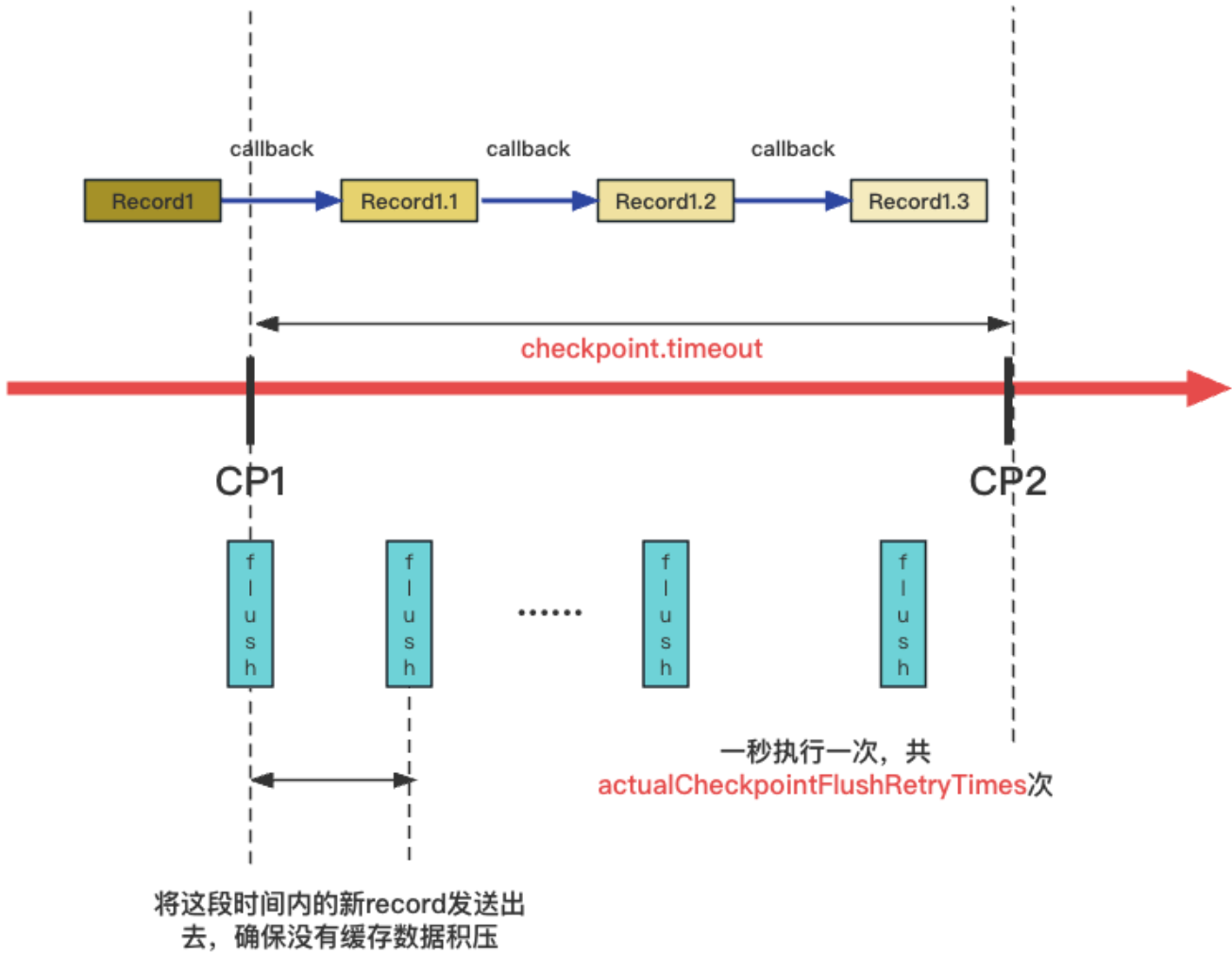
// 该构造函数会将schema包装为KafkaSerializationSchemaWrapper
public static FlinkKafkaProducer<byte[]> createKafkaProducer(Properties kafkaProps, String outputTopic, FlinkKafkaProducer<byte[]> sink = new FlinkKafkaProducer<>(outputTopic,
    (SerializationSchema<byte[]>) (element) -> element,
    kafkaProps,
    partitioner,
    semantic,
    5
);
```

二、有关kafka参数和RetryableProducerCallback参数规范

下图红色时间轴为Flink时间，模拟一个checkPoint完整执行过程。

时间轴上为FlinkKafkaProducer中invoke主函数执行过程，对于Record1这里考虑极端情况，在CP1开始时，Kafka Broker不可用。首先该发送函数线程，会等待**delivery.timeout.ms**时间，在超时失败后触发回调函数，选择其它partition，考虑极端情况会重试**actualCallbackRetryTimes**次。

时间轴下方FlinkKafkaProducer中snapShot中执行流程，会每隔1s进行一次flush动作，除了第一个flush动作时将CP1前所有数据确保发送完毕，后面的flush动作是把CP执行过程中，callback中重发的数据进行一次全量发送下游。执行次数由**actualCheckpointFlushRetryTimes**指定。



参数合理性设置

为了保证所有流程严格正确执行，参数设置必须符合下面要求

$$\text{actualCallbackRetryTimes} * \text{delivery.timeout.ms} < \text{actualCallbackRetryTimes} < \text{checkpoint.timeoout} * \text{【factor: default0.8】}$$

下面以RTLTP为例

- actualCallbackRetryTimes 默认值3
- actualCallbackRetryTimes 默认值120
- delivery.timeout.ms kafka自身默认120,000 可以缩小为30,000
- checkpoint.timeoout rtlp设置为60s 可以改为180s

此时考虑到最坏情况. $90s < 120 < 180s * 0.8$, 整个流程能够确保完整运行。

数据完整性测试

一、前期数据准备

以rtlp-fee任务为例，测试下游写出dsp.charge.6.click在该topic的一个broker掉线后情况。

上游topic	定量分析数据量	写出topic
test.d.vc.6.m	634116	testdsp.charge.6.click

consumer_casino-test_dup_based_on_test

Consumed topic information		
Topic	Partitions Covered %	Total Lag
test.d.vc.6.m	100	634116

消费者Topic: test.d.vc.6.m 配置如下，单副本，使用broker2、3、4。

test.d.vc.6.m

Topic Summary

Replication	1
Number of Partitions	6
Sum of partition offsets	490,066,792
Total number of Brokers	4
Number of Brokers for Topic	3

Partitions by Broker

Broker	# of Partitions	# as Leader	Partitions	Skewed?	Leader Skewed?
2	2	2	(2,5)	false	false
3	2	2	(1,4)	false	false
4	2	2	(0,3)	false	false

生产者Topic: testdsp.charge.6.click 的配置如下，单副本，使用broker2、3、4。

testdsp.charge.6.click

Topic Summary

Replication	1
Number of Partitions	6
Sum of partition offsets	1,613,289,444
Total number of Brokers	4
Number of Brokers for Topic	3

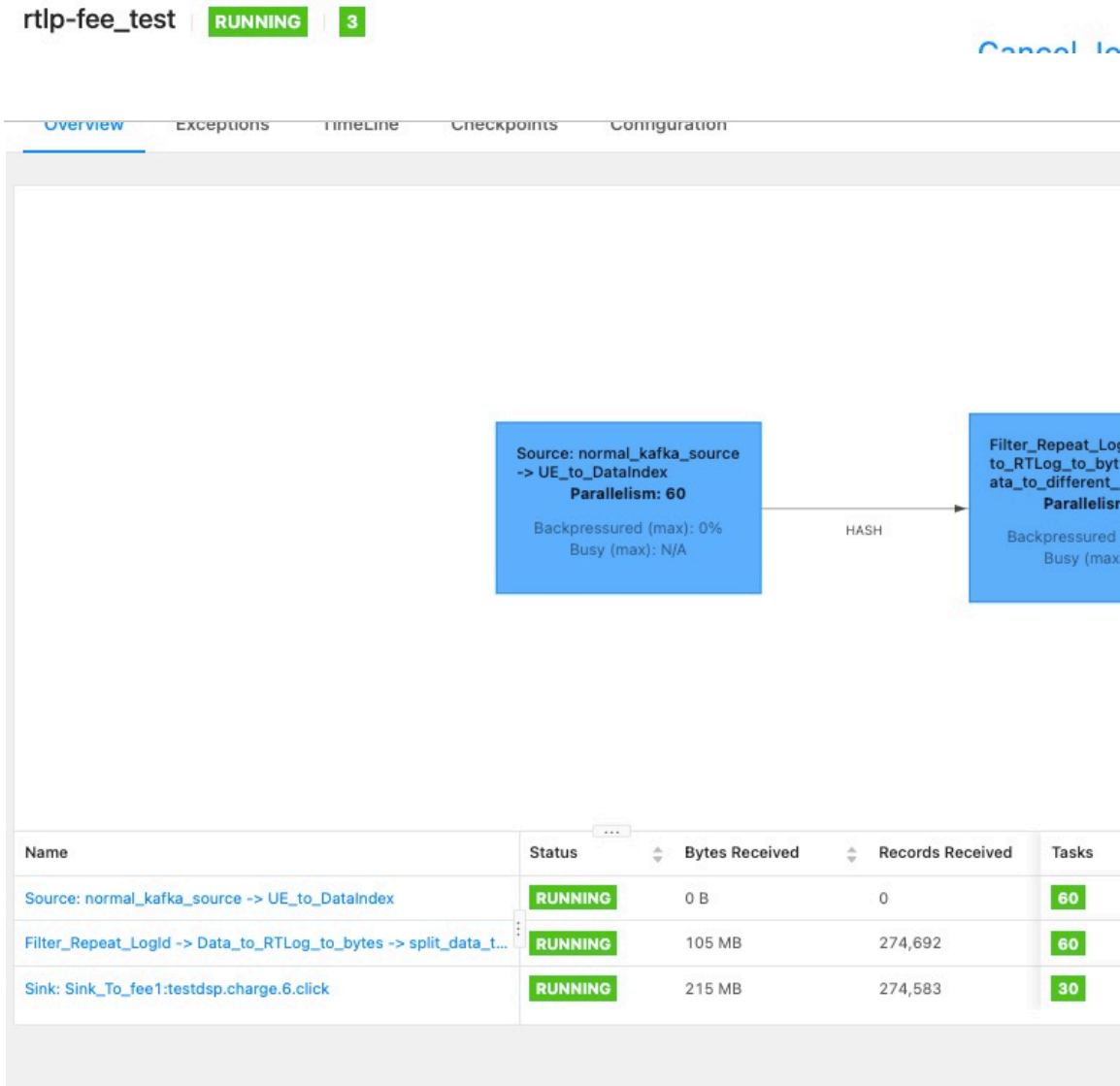
Partitions by Broker

Broker	# of Partitions	# as Leader	Partitions	Skewed?	Leader Skewed?
2	2	2	(2,5)	false	false
3	2	2	(0,3)	false	false
4	2	2	(1,4)	false	false

二、启动Job 开始模拟Flink正常运行的任务

三、关闭broker2

2023-08-23 09:56:26 当前Job已经消费完27w条数据



此时进入broker2的机器中，立刻kill 当前kafka进程

```
root@kafka:/opt/kafka/bin# ./kafka-topics.sh --zookeeper.connect=zoo1:2181 --zookeeper.connect=zoo2:2181 --zookeeper.connect=zoo3:2181 --list
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$ sudo kill 31479
[sudo] zhuming3 的密码:
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$ echo $(date)
2023年 08月 23日 星期三 09:59:37 CST
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$
```

从日志中可以看到，在kill掉kf322ss这台机器后，kafka-producer发送线程已经报警，显示broker连接不上

```
2077 2023-08-23 09:58:05,360 [Sink: Sink_To_fee1:testdsp.charge.6.click (22/30)#0] INFO
org.apache.flink.streaming.api.functions.sink.TwoPhaseCommitSinkFunction [] - FlinkKafkaProducer 22/30 -
checkpoint 1 complete, committing transaction TransactionHolder{handle=KafkaTransactionState
[transactionalId=null, producerId=-1, epoch=-1], transactionStartTime=1692755797275} from checkpoint 1
2078 2023-08-23 09:59:03,489 [kafka-producer-network-thread | producer-5] WARN
org.apache.kafka.clients.NetworkClient [] - [Producer clientId=producer-5] Connection
to node 2 (kf322ss.jx.shbt2.qihoo.net/10.132.112.201:9092) could not be established. Broker may not be
available.
2079 2023-08-23 09:59:03,489 [kafka-producer-network-thread | producer-4] WARN
org.apache.kafka.clients.NetworkClient [] - [Producer clientId=producer-4] Connection
to node 2 (kf322ss.jx.shbt2.qihoo.net/10.132.112.201:9092) could not be established. Broker may not be
available.
2080 2023-08-23 09:59:03,498 [kafka-producer-network-thread | producer-1] WARN
org.apache.kafka.clients.NetworkClient [] - [Producer clientId=producer-1] Connection
to node 2 (kf322ss.jx.shbt2.qihoo.net/10.132.112.201:9092) could not be established. Broker may not be
available.
2081 2023-08-23 09:59:03,498 [kafka-producer-network-thread | producer-2] WARN
```

在过去delivery.timeout.ms 时间后，发送超时，进入自定义的回调函数中。下面日子为RetryableProducerCallback中测试写出的日志。partition5在broker2中，重新选择partition 0写出。

```
available.
2114 2023-08-23 10:00:03,437 [kafka-producer-network-thread | producer-5] ERROR
2115 2023-08-23 10:00:03,438 [kafka-producer-network-thread | producer-5] ERROR
net.qihoo.ads.flink.kafka.RetryableProducerCallback [] - try re send error kafka record, old
partition = 5, new partition = 0
2116 2023-08-23 10:00:03,438 [kafka-producer-network-thread | producer-5] ERROR
net.qihoo.ads.flink.kafka.RetryableProducerCallback [] - try re send error kafka record, old
partition = 5, new partition = 0
2117 2023-08-23 10:00:03,438 [kafka-producer-network-thread | producer-5] ERROR
net.qihoo.ads.flink.kafka.RetryableProducerCallback [] - try re send error kafka record, old
partition = 5, new partition = 0
2118 2023-08-23 10:00:03,438 [kafka-producer-network-thread | producer-5] ERROR
net.qihoo.ads.flink.kafka.RetryableProducerCallback [] - try re send error kafka record, old
partition = 5, new partition = 0
```

四、屏蔽坏partition后稳定状态

消费者**Topic: test.d.vc.6.m** 除了partition2和5因为broker2宕机，无法消费，其它partition数据消费完成。

➤ consumer_casino-test_dup_based_on_test / test.d.vc.6.m

Topic Summary	
Total Lag	118,309
% of Partitions assigned to a consumer instance	100

test.d.vc.6.m				
Partition	LogSize	Consumer Offset	Lag	Consumer Instance Owner
0	81,910,601	81,910,601	0	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249
1	81,542,202	81,542,202	0	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249
2	81,571,553	81,512,415	59,138	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249
3	81,916,013	81,916,013	0	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249
4	81,580,601	81,580,601	0	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249
5	81,545,822	81,486,651	59,171	consumer-consumer_casino-test_dup_based_on_test-1-a568aaef-a774-44e4-986f-ff2bdadaa883;/10.254.171.249

生产者**Topic: testdsp.charge.6.click** 的partiton2和5因为broker2宕机，无法写入，平均分散写入其它partition

consumer_casino-test_dup_based_rtlp_test / testdsp.charge.6.click

Total Lag	515,068
% of Partitions assigned to a consumer instance	100

testdsp.charge.6.click

Partition	LogSize	Consumer Offset	Lag	Consumer Instance Owner
0	269,285,024	269,178,862	106,162	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
1	269,092,476	268,986,293	106,183	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
2	268,305,013	268,259,553	45,460	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
3	269,210,863	269,105,195	105,668	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
4	269,060,598	268,954,517	106,081	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
5	268,216,422	268,170,908	45,514	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249

五、恢复broker2的kafka进程

```
随后恢复broker的kafka进程
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$ echo $(date)
2023年 08月 23日 星期三 10:06:20 CST
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$ sudo ./kafka-start.sh
starting Kafka
MVAD: [zhuming3@kf322ss] /opt/kafka/bin$
```

在过去一段时间后所有剩余数据均匀写入6个partition中

consumer_casino-test_dup_based_rtlp_test / testdsp.charge.6.click

Topic Summary

Total Lag	634,116
% of Partitions assigned to a consumer instance	100

testdsp.charge.6.click

Partition	LogSize	Consumer Offset	Lag	Consumer Instance Owner
0	269,304,743	269,178,862	125,881	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
1	269,112,255	268,986,293	125,962	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
2	268,325,023	268,259,553	65,470	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
3	269,230,583	269,105,195	125,388	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
4	269,080,231	268,954,517	125,714	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249
5	268,236,609	268,170,908	65,701	consumer-consumer_casino-test_dup_based_rtlp_test-1-d738a9b8-7bc9-4ae8-9fcd-bd72fac7a4c9;/10.254.171.249

对于这些数据用hashset计算logid的总数，可以发现数据同样是634,116，没有重复和缺失。

```
634114
634115
634116
```

无标签